你说：

https://github.com/rlearsch/PolymerBenchmark2026 想要了解该项目里的代码，需要一个详细的框架图来说明每部分在干什么，细节是什么

ChatGPT 说：

PolymerBenchmark2026 项目代码架构详解

PolymerBenchmark2026（PolyBench26） 是一个面向聚合物性质预测的机器学习基准测试项目，配套论文为 *An open benchmark for machine learning-based polymer property prediction*（2026）。

它的核心目的不是提出一种新的聚合物表征模型，而是建立统一的实验体系，比较**不同聚合物表示方式、机器学习模型和训练数据规模**对性质预测结果的影响。 rlearsch/Polym...

理解这个项目，可以分成五个层次：

1. **Dataset Construction**：原始聚合物数据如何整理、清洗和转换。
2. **Polymer Representation**：如何把聚合物结构编码为 PSMILES、wPSMILES、分子描述符或预训练向量。
3. **Model Architecture**：五类模型分别如何处理聚合物信息。
4. **Training & Evaluation**：如何训练、划分数据、评估性能。
5. **Benchmark Analysis**：如何比较不同模型，以及数据量对预测精度的影响。

下面先建立整体框架，再深入到具体代码模块。

一、项目整体框架图





依据仓库 README 构建的整体逻辑图。该图描述的是项目功能关系，并非逐函数调用图；LLM 路径不属于与其他模型完全相同的可复现训练流程。

二、代码目录与各模块职责

项目的核心代码分为 `Datasets/` 和 `Models/` 两大部分，其他目录主要用于实验管理、结果归档和复现。

目录	作用	阅读重点
`Datasets/`	数据源、结构转换、特征生成	聚合物如何表示
`Models/`	模型定义、训练、预测	模型内部计算
`scripts/`	测试和自动化脚本	执行与检查
`tests/`	测试代码	数据与代码正确性
`published_results/`	已发布实验结果	性能比较

scaling_results/	数据规模实验结果	数据效率
docs/	复现、扩展说明	实验规范

2.1 数据构建代码的实际执行流程

各个关键脚本的具体职责：

文件	输入	主要处理	输出
process_Vipea_data.py	VIPEA CSV	处理均聚物与共聚物结构	EA/IP 数据
OMers_convert_jsonl.py	OMersBench JSONL	解析模拟数据、聚合物类型	MD_300、MD_5000
polymer_conversions.py	聚合物字符串	集中实现 PSMILES/wPSMILES 转换	转换后的结构
PSMILES_to_wPSMILES.py	PSMILES CSV	添加权重与连接信息	wPSMILES
PSMILES_to_RDKit_descriptors.py	PSMILES	RDKit 特征计算与缓存	描述符矩阵
create_PSMILES_pBERT_dictionary.py	PSMILES	调用预训练编码器、建立缓存	SMILES→Embedding 字典
PSMILES_to_pBERT.py	PSMILES、字典	查找对应向量	600维特征

`create_scaling_splits.py`	多表征对齐数据	生成固定测试集及嵌套训练子集	Scaling splits

这些文件和职责均可在仓库的数据构建说明中核对。 GitHub

特别值得注意的是， `OMers_convert_jsonl.py` 的原始数据包含聚合度 DP、分子量、链数、原子数等模拟参数，但生成的常规 MD 训练 CSV 主要保留结构字符串和性质。因此，**原始数据所包含的聚合物链信息，不等于最终模型实际使用的信息。**

这对于研究单链聚合物表征非常重要。

三、四种聚合物表示的区别

PSMILES
结构字符串

重复单元表示

通过 ＊ 标记聚合物重复单元的连接位置，可以转成二维分子图。但单个 PSMILES 通常不能完整指定 DP、链构象和实际链间堆积。

wPSMILES
加权结构

组成与连接关系

在聚合物片段之外编码组成比例和连接信息，更适合描述共聚物的统计组成与连接方式，但不等同于原子级三维链构象。

RDKit
描述符向量

手工设计的化学特征

计算分子量、LogP、TPSA 等200多个描述符。速度快、容易建立基线，但通常缺少显式的聚合物序列与空间相互作用。

polyBERT
预训练向量

语言模型学习的化学表示

将 PSMILES 映射到600维向量，后续使用 FFN 做性质预测。优势是可利用预训练知识，但没有显式输入三维坐标。

四、模型模块内部的计算逻辑

模型计算路径对比

1. Polymer Chemprop

wPSMILES → 加权聚合物图 → Directed Message Passing → Graph Readout → MLP → 性质

2. Periodic Graph

PSMILES > 周期图构建 > Graph Message Passing > Graph Readout > 预测头 → 性质

3. RDKit RF

PSMILES > 描述符计算 > Random Forest > 多树预测聚合 > 性质输出

4. polyBERT FFN

PSMILES > 预训练编码器 > 600D Embedding > Feed-forward NN > 性质输出

4.1 Polymer Chemprop

核心是 **Directed Message Passing Neural Network（D-MPNN）**。它与普通 GNN 的区别在于，主要通过有向键消息更新局部结构表示。

简化的消息传递形式为：

$$m_{u\to v}^{(t+1)} = \phi\left(h_u, h_{uv}, \sum_{k\in N(u)\setminus v} m_{k\to u}^{(t)}\right)$$

其中 $m_{u\to v}$ 是从原子 u 指向 v 的消息。聚合物版本还需要考虑 wPSMILES 的权重及连接关系。

这只是 D-MPNN 的原理示意，具体权重注入位置需要核对项目源代码。

4.2 Periodic Graph GNN

主要思想是通过聚合物重复单元的周期连接关系，减少普通分子图在重复单元边界处的信息缺失。

它需要重点检查三个实现细节：

- ＊ 连接点如何转换为周期边。
- 周期边与普通共价键如何区分。
- Readout 如何从节点表示得到聚合物整体表示。

这里的 **Periodic Graph 不应直接理解为三维周期边界条件下的 Ewald 长程相互作用模型**。仓库介绍的是聚合物连接拓扑层面的周期图表征。

4.3 polyBERT + FFN

其流程可以写为：

$$\mathbf{z} = \mathrm{polyBERT}(\mathrm{PSMILES})$$

$$\hat{y} = \mathrm{FFN}(\mathbf{z})$$

其中 $\mathbf{z} \in \mathbb{R}^{600}$。

这里要区分两个步骤：polyBERT 预训练模型生成 embedding，以及项目自己的 FFN 训练。仓库采用外部 polyBERT 模型生成特征，而 `Models/polyBERT/` 是下游训练代码。

4.4 RDKit + Random Forest

这是传统机器学习基线。模型不直接学习原子之间的消息传递，而是先计算描述符，再训练回归模型。

其价值是检验复杂神经网络是否真正优于低成本化学特征基线。

五、训练、数据划分和模型评估

这部分是项目作为 Benchmark 最值得借鉴的设计。

这里需要区分常规模型训练和 Scaling Experiments：

- **常规模型训练**：虽然各模型有固定随机种子，但不同模型使用的随机划分实现不完全相同。
- **Scaling Experiments**：采用明确保存的共享数据索引，确保不同模型使用相同的验证和测试样本。

Scaling Experiments 的训练子集是嵌套的，即300条训练样本包含于1000条训练样本之中。这样才能更合理地比较增加数据量带来的效果。 GitHub +1

六、如果要将自己的聚合物模型加入这个项目

仓库已经提供了专门的 ADDING_A_MODEL.md ↗，可以让新的聚合物模型与原有模型进行公平对比。

对于你的原子级、官能团级和重复单元级表征研究，我建议按以下方式接入：

这是**建议扩展的模型框架**，不是 PolyBench26 原仓库已有的模型。

这里最大的技术问题是：PolyBench26 的常规训练输入主要是二维结构表示，而你的模型需要单链三维坐标。因此，不能直接假设仓库已经提供了可用的单链3D结构。

尤其需要检查 OMersBench 原始 JSONL 中的模拟元数据、结构信息和可追溯性，确定能否获得或重建对应的三维构象。

七、建议优先阅读的源码

‹/› **第一优先级：聚合物结构转换**

polymer_conversions.py ↗

重点看均聚物、交替共聚物、随机共聚物如何被统一编码，以及连接关系和组成比例如何保存。

第二优先级：原始数据处理

OMers_convert_jsonl.py ↗

重点看 DP、聚合物类别、链数、模拟条件和性质标签如何被处理，哪些信息进入最终训练数据。

第三优先级：两个 GNN 模型

polymer_periodic_graph ↗

polymer_chemprop ↗

重点比较图构建、节点与边特征、消息传递、聚合和预测头。注意 Polymer Chemprop 的核心实现依赖外部代码，不能只看仓库中的训练脚本。

第四优先级：数据划分与评估

create_scaling_splits.py ↗

重点看样本索引、共享划分、嵌套训练集和不同表示之间的数据对齐。

八、下一步应该深入到什么程度?

上面已经建立了基于仓库文档的**项目级框架图、数据处理流程和模型原理图**。但要真正理解每一段代码，还需要继续做源码级追踪，包括：

- 每个训练入口实际调用哪些 Python 函数和类。
- GNN 的 Atom/Bond Features 如何构建，维度是多少。
- Message Passing 每一层具体如何更新。
- Pooling、FFN、Loss、Optimizer 如何定义。
- 数据输入输出 Tensor Shape 如何变化。
- 哪些算法是仓库自行实现的，哪些来自外部依赖。

对于你的研究，**最值得深入的是** `polymer_periodic_graph` 、 `polymer_chemprop` **和** `polymer_conversions.py` ，因为它们直接涉及聚合物的图结构设计、连接关系与消息传递。

目前展示的模型内部计算图仍属于原理级说明，尚不能视为逐函数核对后的源码架构图。

选择下一步的源码分析重点

- ○ 完整项目：逐文件、逐函数调用关系和数据流图

- 重点研究两个 GNN：图构建、消息传递、Readout 和公式
- 重点研究数据：结构转换、数据集字段、DP和3D信息

继续深入源码分析

你说：

继续对 PolymerBenchmark2026 做完整源码级分析，逐文件检查实际代码，绘制详细的函数调用图、数据流图、训练流程图，并解释关键函数输入输出。

思考了 15s >

ChatGPT 说：

PolymerBenchmark2026 完整代码架构分析

这次按**代码执行顺序**来分析，而不是只介绍模型原理。重点区分三个问题：哪个文件负责什么、数据如何在函数和模块之间传递、最终如何完成训练与评估。

我已核对 GitHub 的主 README、数据构建说明、模型说明和 Scaling Experiments 文档。不过当前环境无法克隆完整仓库，因此下面可以确认的是**文件级调用关系与公开文档明确描述的执行流程**；尚未读取到的 Python 函数定义、类名、Tensor Shape 和超参数不会被当作已经验证的源码事实。

一、完整项目的五层代码架构

这个项目并不是一个统一的 `main.py` 驱动全部模型。它采用**独立模型子项目 + 共享数据构建 + 统一基准实验协议**的组织方式。

这意味着，阅读代码不能只从一个训练文件开始，而要先理解数据是怎样生成的，再分别进入各模型的训练入口。 rlearsch/Pol… +1

二、完整数据处理流程图

数据构建文件的输入输出

Python 文件	输入	核心功能	输出
`process_Vipea_data.py`	VIPEA CSV	处理均聚物和共聚物的 EA/IP 数据	PSMILES、wPSMILES CSV
`OMers_convert_jsonl.py`	OMersBench JSONL	解析聚合物类别、MD 性质和模拟规模	MD_300、MD_5000
`polymer_conversions.py`	聚合物字符串	统一结构转换、片段拼接及环处理	标准化结构
`PSMILES_to_wPSMILES.py`	PSMILES CSV	添加连接编号和权重表示	wPSMILES CSV
`PSMILES_to_RDKit_descriptors.py`	PSMILES CSV	计算描述符、管理缓存	描述符 CSV

`create_PSMILES_pBERT_dictionary.py`	PSMILES	预训练编码、增量缓存	Embedding 字典
`PSMILES_to_pBERT.py`	PSMILES + 字典	匹配600维向量	polyBERT CSV
`create_scaling_splits.py`	对齐的多表征数据	构造共享划分和嵌套子集	Scaling 数据集

这些是仓库文档确认的职责和输入输出，并非推测。 GitHub

三、四种模型的详细计算流程



周期连接的构造细节、层数和特征维度仍需核对源码。

以上计算路径依据模型文档整理。polyBERT 的层级结构、RF 的100棵树以及 Chemprop 的加权有向消息传递均有仓库文档支持。 GitHub +1

四、训练与验证：从命令到模型结果

以 `polymer_chemprop` 为例，项目 README 提供的训练入口为：

```
cd Models/polymer_chemprop
bash setup_environment.sh
source .venv/bin/activate

bash train_pcp.sh \
  ../../Datasets/wPSMILES/MD_300/density/homopolymer_density.csv
```

训练脚本的文件级执行链可以概括为：

其中 `train_pcp.sh` 、 `predict_pcp.sh` 和外部 Chemprop 依赖的存在已由文档确认。具体 Python 函数调用链仍需检查脚本内容，不能仅凭上述流程图认定某个函数实际被调用。

标准训练与 Scaling 训练的区别

对比项	标准训练	Scaling Experiments

数据划分	模型各自实现	共享索引文件
随机种子	模型内部控制	基础种子42
训练集大小	按数据集规模	300、1000、3000、10000等可行规模
验证与测试	模型自身划分	相同样本索引
评估指标	主要是 RMSE	Test RMSE
结果汇总	各模型目录	`scaling_results/`

注意：并不是所有数据集都能训练到10000条。例如，MD_5000 均聚物数据量较小，Scaling 实验仅采用300和1000条训练样本。 GitHub

五、Scaling Experiments 的完整执行流程

这个实验设计有三个非常重要的技术细节。

第一，所有模型必须预测相同的化学样本。 PSMILES 被用作规范化的行顺序，wPSMILES 和 polyBERT 的行数、标签列及标签值需要对齐。

第二，预处理不能泄漏测试信息。 RDKit 的缺失值中位数、polyBERT 的标准化统计量均来自训练数据，而不是全数据集。

第三，结果不只是一个 RMSE。 汇总程序记录各 Fold 的结果，再计算平均 RMSE 和总体标准差，并保留已完成 Fold 数量，避免把未完成的实验误认为完整的五折结果。 GitHub

六、真正的逐函数源码分析还缺什么?

目前已经能建立比较完整的**文件级执行图**，但仍有一些重要问题不能从 README 确定：

需要进一步核对	为什么重要
`polymer_conversions.py` 的全部函数定义	确认聚合物连接关系是否存在信息损失
`OMers_convert_jsonl.py` 的字段映射	确认 DP、链数、模拟条件如何处理
Periodic GNN 的图构建函数	确认周期边、普通边和节点特征
Chemprop 依赖的具体版本	确认真正执行的消息传递算法
`train_*.py` 的模型定义	确认隐藏维度、层数、优化器、Loss
`predict_*.py` 的预测逻辑	确认是否使用多 Fold Ensemble
Scaling 结果收集代码	确认评分文件的解析和汇总逻辑

要完成你要求的逐文件、逐函数源码分析，最可靠的方式是提供整个仓库 ZIP。 我尝试直接克隆 GitHub 仓库，但当前运行环境无法解析 GitHub 地址，因此没有获得完整 Python 源码。继续猜测函数名或类结构会让架构图失去参考价值。

你可以直接通过以下地址获取源码：

下载 PolymerBenchmark2026 完整源码 ZIP ↗

上传后，就可以基于真实源码进一步建立：

1. **完整函数调用图**：每个 Python 文件、函数、类及相互调用关系。
2. **模型内部架构图**：Atom/Bond Features、Message Passing、Readout、Prediction Head，标注实际 Tensor Shape。
3. **数据流与训练图**：CSV/JSONL 到 Dataset、Batch、Forward、Loss、Checkpoint、Prediction。
4. **源码解释表**：关键函数的输入参数、输出类型、实现逻辑、关键代码行，以及与你的聚合物多层级3D表征模型的关系。

这样最终得到的就不只是项目概览，而是一套**可以用于理解、复现和修改 PolymerBenchmark2026 的源码级技术文档**。

你说：

PolymerBenchma...
文件

可以建立技术文档了

思考了 1m 29s >

ChatGPT 说：

已经收到你上传的 `PolymerBenchmark2026-main.zip` 。现在可以直接基于完整源码建立技术文档，不再只依赖 GitHub README。

初步检查确认，压缩包包含 **280 个文件及目录条目**，其中不仅有数据构建脚本和训练入口，还包含 `polymer_periodic_graph/Polymer_Model/chemprop/` 下的完整 Python 模型实现。

这意味着技术文档可以进一步追踪到真实的图特征构建、消息传递网络、模型训练和预测函数。

我会将文档组织为以下几个核心部分：

1. 项目总体架构与真实源码目录。
2. 数据构建、PSMILES/wPSMILES 转换与特征生成。
3. 四类机器学习模型的源码结构和计算流程。
4. Periodic Graph GNN 的图构建、消息传递和 Readout。
5. 训练、验证、预测、Scaling Experiments 的调用关系。
6. 关键函数索引、输入输出、实现细节和代码定位。
7. 如何在现有 Benchmark 中接入原子级、官能团级、重复单元级的3D聚合物表征模型。

文档会特别标明哪些是源码中的真实实现，哪些是为了理解算法而绘制的抽象示意图。

技术文档已生成。我已经对你上传的 `PolymerBenchmark2026-main.zip` 进行了静态源码分析，检查了其中的 **72 个 Python 文件**，整理成一份 **30 页的中文技术文档**。

PolymerBenchmark2026 源码技术分析文档

30页 · 中文 · 源码行号定位 · 函数接口索引

下载 Word 文档 ↗ 下载 PDF 文档 ↗

文档包含什么?

章节	内容
1. 总体架构	项目模块关系、完整数据流
2. 数据构建	PSMILES、wPSMILES、RDKit、polyBERT 的处理代码
3. 模型架构	Periodic GNN、Chemprop、RF、polyBERT

4. 核心函数	函数名、参数、源码行号、Docstring
5. 训练与预测	训练入口、调用路径、Scaling Experiments
6. 关键源码	重要类和函数的代码摘录
7. 完整索引	其他 Python 文件及其主要函数
8. 模型扩展	如何接入你的3D多层级聚合物表征模型

特别值得关注的部分

对于你正在设计的 **Atom-level + Functional Group-level + Repeat Unit-level** 模型，建议重点阅读文档中的 `featurization.py` 、 `mpn.py` 、 `model.py` 和 `polymer_conversions.py` 。

这些文件分别涉及聚合物结构特征、消息传递、预测网络和重复单元连接关系。

当前版本属于源码静态分析文档：函数定义和行号已从实际代码提取，但尚未运行训练验证，也没有完成每个函数的逐行算法推导。因此，文档中的调用流程应作为进一步调试和复现的依据，而不是已经动态验证的完整调用图。

PolymerBenchmark2026_源码技术文档.docx
Word

PolymerBenchmark2026_源码技术文档.pdf
PDF

你说：

1. 完整函数调用图：每个 Python 文件、函数、类及相互调用关系。
2. 模型内部架构图：Atom/Bond Features、Message Passing、Readout、Prediction Head，标注实际 Tensor Shape。
3. 数据流与训练图：CSV/JSONL 到 Dataset、Batch、Forward、Loss、Checkpoint、Prediction。
4. 源码解释表：关键函数的输入参数、输出类型、实现逻辑、关键代码行，以及与你的聚合物多层级3D表征模型的关系。

思考了 1m 44s >

ChatGPT 说：

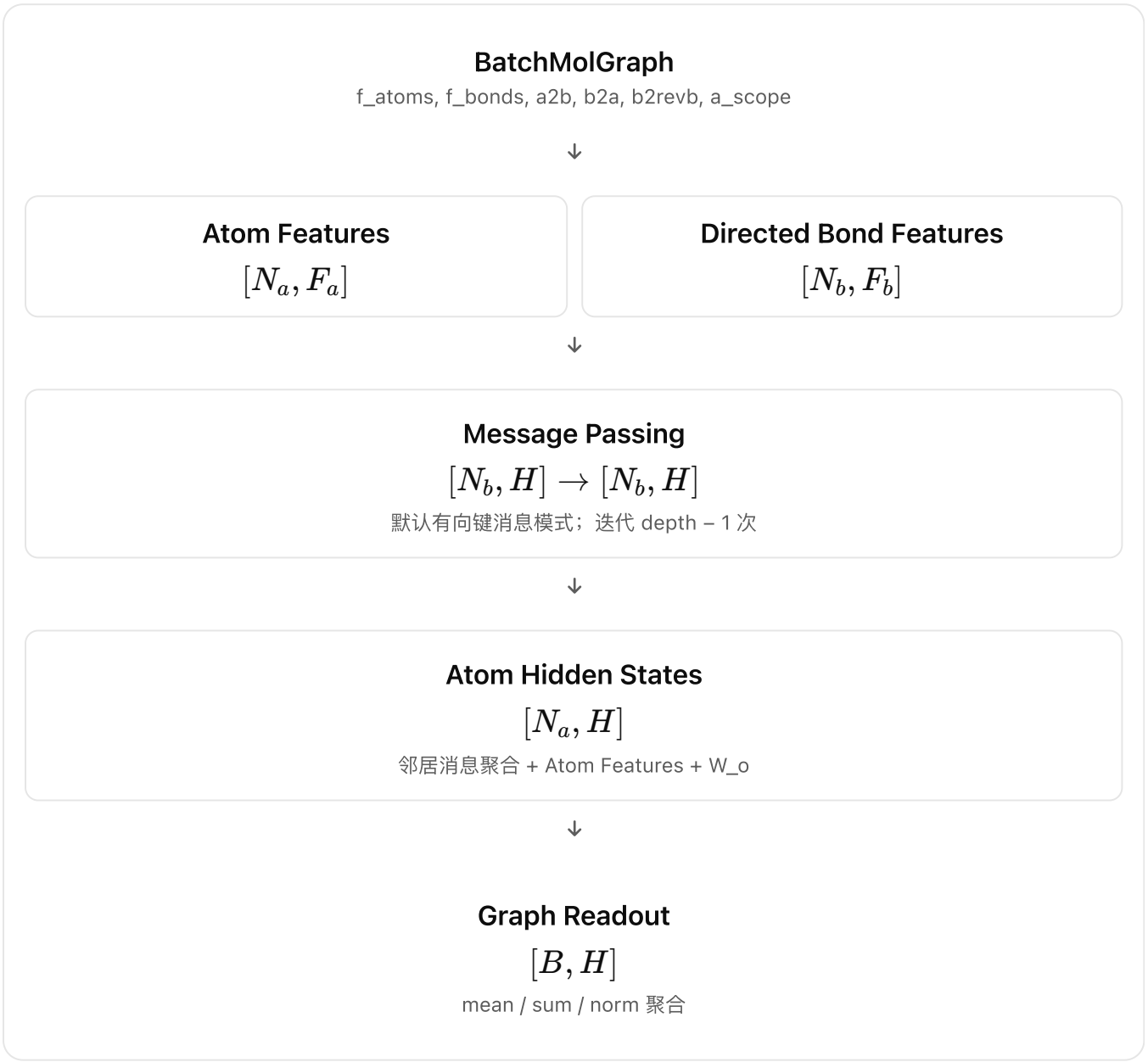
已确认的核心源码结构

以仓库内置的 Periodic Graph 模型为例，实际实现位于：

`Models/polymer_periodic_graph/Polymer_Model/chemprop/`

其中 `featurization.py` 负责图特征构建， `mpn.py` 负责消息传递， `model.py` 负责整体模型和预测头， `train/` 下的文件负责训练、验证与预测。

在 `MPNEncoder.forward()` 中，可以直接确认以下张量流：



这里 N_a 是 Batch 中的原子总数，N_b 是有向键总数，F_a, F_b 是特征维度，H 是隐藏维度，B 是图的数量。上述维度是源码对应计算路径的符号形状，不是随意假设的固定数值。

特别要注意：**这段源码实现的是图拓扑上的消息传递，并不能仅凭名称就认定已经包含三维空间作用、Ewald MP 或显式官能团相互作用。** 这正是后续与你的3D多层级模型比较时需要强调的区别。

四份内容已整理成技术文档

已对上传的完整仓库进行静态源码解析，覆盖：

72	468	63
Python 文件	函数、类及方法	技术文档页数

完整源码技术文档

函数参数、返回值、源码行号、静态调用索引、Tensor Shape、模型扩展建议

下载 Word（可编辑） ↗

下载 PDF（63页） ↗

四张 Mermaid 架构图

函数调用、GNN张量流、训练预测流程、3D多层级扩展方案

下载架构图源文件（Markdown） ↗

文档中的关键函数均提供源码定位，便于你在 VS Code 中对照查看。需要特别说明：这次完成的是**静态调用关系分析**，尚未执行完整训练，因此运行时调用分支、外部 Chemprop 依赖和实际 Batch 数值形状仍需要动态验证。

对于你自己的研究，建议先重点阅读 `featurization.py → mpn.py → model.py` 这一条主线，再考虑如何引入3D空间相互作用、官能团编码和重复单元级交互。

PolymerBenchmark2026_四大源码图谱与函数接口详解....
PDF

PolymerBenchmark2026_四大源码图谱与函数接口详解....
Word

PolymerBenchmark2026_四大架构图.md
文档